BRIDGE CAPACITY MANAGEMENT SYSTEM

PROGRESS REPORT PROYEK I WI1102 COMPUTATIONAL THINKING

THE STRUCTURES WE CROSS WITHOUT A THOUGHT


Beryl Raditya Ahmad


Fawwaz M. Zaahiy


Muhammad Hisyam M.R. Arisyia


Tristan Pudyameka Sakti


Muhammad Alif Fahmi Jati

LATAR BELAKANG & TUJUAN

LATAR BELAKANG

Jembatan merupakan infrastruktur vital dengan batas kapasitas beban tertentu, sehingga kelebihan muatan berisiko menimbulkan kerusakan struktural hingga keruntuhan, terutama pada kondisi lalu lintas dua arah yang sulit dipantau secara langsung. Untuk mengatasi tantangan tersebut, Bridge Capacity Management System (BCMS) hadir sebagai solusi simulasi berbasis CLI yang memodelkan manajemen beban lalu lintas dan memvisualisasikan pergerakan kendaraan dua arah secara real-time.



TUJUAN

1. Mengimplementasikan konsep-konsep dasar pemrograman (sekuens, kondisional, loop, array, dan sub-program) dalam sebuah simulasi nyata.
2. Menerapkan empat pilar Computational Thinking (Dekomposisi, Pengenalan Pola, Abstraksi, dan Algoritma) dalam perancangan sistem.
3. Membangun simulasi FSM untuk mendeteksi status keselamatan jembatan secara dinamis.
4. Memvisualisasikan proses manajemen beban jembatan melalui antarmuka CLI berbasis ASCII.

CARA KERJA

ALUR MANAJEMEN BEBAN LALU LINTAS DUA ARAH SECARA OTOMATIS BERBASIS TIME-DRIVEN SIMULATION.



VEHICLE ARRIVAL

Kendaraan muncul secara acak dari Jalur A (kiri) dan Jalur B (kanan) secara simultan sesuai probabilitas stokastik.



DUAL-LANE MOVEMENT

Kendaraan melintas pada array terpisah (Jalur A sisi kanan, Jalur B sisi kiri dengan arah berbeda) disertai kalkulasi akumulasi fatigue struktur jembatan.



WEIGHT CHECK & FSM

Sistem menghitung total beban statis. Jika total beban melebihi 100 ton (status kritis), kendaraan ditolak (palang ditutup) atau masuk ke antrean FIFO.



LIVE CLI RENDERING

Sistem memperbarui tampilan terminal pada 10 FPS tanpa efek berkedip dan otomatis mengurangi beban saat kendaraan keluar dari jembatan.

ABSTRAKSI

01 MODEL STRUKTUR JEMBATAN

MEMPRESENTASIKAN JEMBATAN FISIK MENJADI STRUKTUR OBJEK DIGITAL MELALUI KELAS OOP DENGAN ATRIBUT BEBAN DAN KAPASITAS

02 KARAKTERISTIK KENDARAAN

MENYEDERHANAKAN VARIASI RAGAM KENDARAAN DUNIA NYATA KE DALAM KATEGORI STANDAR BERAT STATIS (MOTOR, MOBIL DAN TRUK)

03 FAKTOR FISIKA DAN KELELAHAN

MENGABSTRAKSIKAN DINAMIKA POSISI TENGAH BENTANG MENJADI MOMEN LENTUR DAN AKUMULASI DEGRADASI MATERIAL FATIGUE.

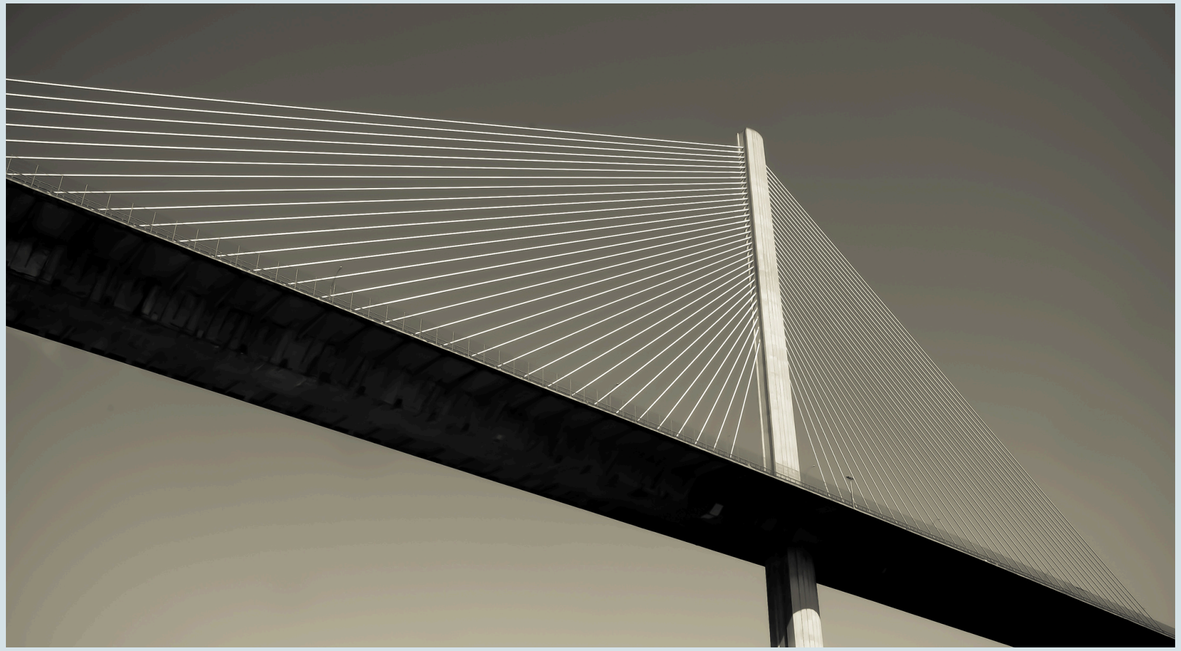
04 STATUS KEAMANAN FSM

MEREDUKSI KOMPLEKSITAS KALUKASI BACKEND MENJADI STATUS OPERASIONAL JELAS: NORMAL, WASPADA, KRITIS, HINGGA PALANG DITUTUP



DEKOMPOSISI MASALAH

EVERY BRIDGE IS A QUIET, CONSTANT NEGOTIATION BETWEEN A FEW BASIC FORCES.



MEMECAH MASALAH MENJADI BAGIAN YANG LEBIH KECIL.

1.MODEL JEMBATAN
BAGAIMANA MENGGAMBARKAN JEMBATAN DI PROGRAM?
GUNAKAN ARRAY (DERETAN KARAKTER) UNTUK 2 JALUR

2.LOGIKA KESELAMATAN
KAPAN JEMBATAN AMAN ATAU BAHAYA?
PAKAI FSM — STATUS BERUBAH OTOMATIS SESUAI BEBAN

3.KEPUTUSAN OTOMATIS
KAPAN KENDARAAN BOLEH MASUK? (KONSEP PALANG PINTU)
CEK BEBAN → TERIMA ATAU TOLAK → YANG DITOLAK MASUK ANTRIAN (APABILA MENCAPAI BATAS KRITIS, PALANG TERTUTUP)

4.TAMPILAN (CLI)
BAGAIMANA MENAMPILKANNYA KE LAYAR?
GAMBAR KOTAK ASCII + POSISI KENDARAAN + STATUS

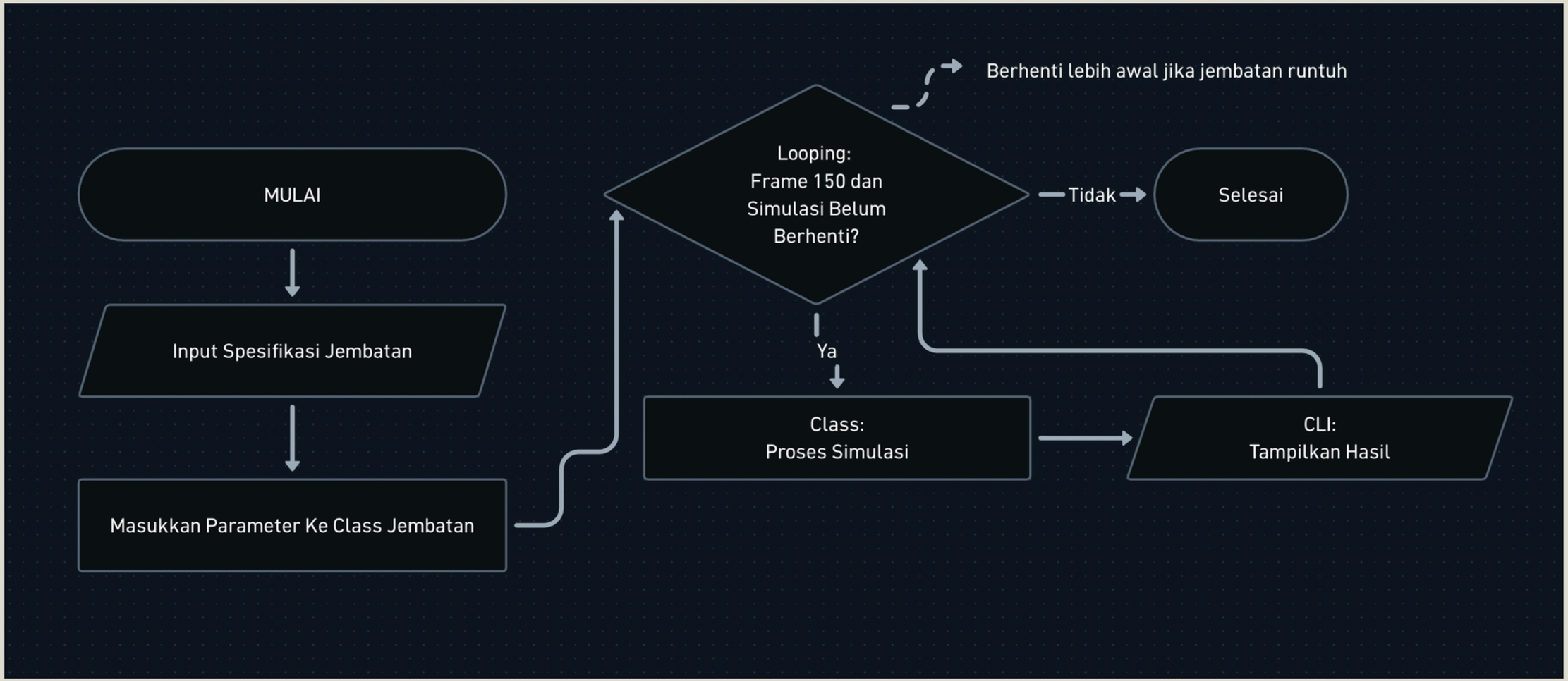
PENGENALAN POLA





Mengidentifikasi ambang batas beban: akumulasi beban di bawah 70% sebagai status Normal, 70%-90% sebagai Waspada, dan di atas 90% sebagai Kritis yang memicu penolakan kendaraan.

PERANCANGAN ALGORITMA



IMPORT

```
#Bagian ini memuat pustaka (library) yang dibutuhkan.
import time #Untuk mengatur kecepatan animasi (jeda)
import random #Untuk stimulasi angka acak
import os #Untuk membersihkan layar agar tampilan tidak menumpuk ke bawah
from rich.live import Live
from rich.layout import Layout
from rich.panel import Panel
from rich.progress import Progress, BarColumn, TextColumn
from rich.table import Table
from rich.text import Text
#Rich untuk membuat tampilan terminal menjadi cantik dan beranimasi. Hanya menggunakan yang diperlukan.
```

LOGIKA INTI TIDAK MENGGUNAKAN LIBRARY.
LIBRARY DIGUNAKAN UNTUK ANIMASI CLI.

OOP

OOP = ATRIBUT (VARIABEL) + METHOD (FUNGSI YANG MENEMPEL PADA OBJEK

OBJECT ORIENTED PROGRAMMING ADALAH JENIS PEMROGRAMAN KOMPUTER DI MANA PROGRAMMER MENDEFINISIKAN TIPE DATA DARI STRUKTUR DATA, DAN JUGA JENIS OPERASI (FUNGSI) YANG DAPAT DITERAPKAN PADA STRUKTUR DATA.

```
# A. ALASAN MENGGUNAKAN OOP (Object-Oriented Programming):
# 1. Abstraksi: Memodelkan entitas fisik jembatan yang kompleks menjadi
#    objek digital (Class) dengan atribut (beban, kapasitas) dan perilaku.
# 2. Enkapsulasi: Mengamankan data state (beban, kesehatan) agar tidak bisa
#    diubah sembarangan dari luar, melainkan hanya melalui method internal.
# 3. Alasan tidak memakai Inheritance/Polymorphism: Untuk menjaga efisiensi
#    dan kesederhanaan kode (KISS), karena kendaraan disimpan sebagai Dictionary.
```

MAIN LOOP

1. INPUT
2. MITIGASI ERROR VALUE
3. MASUKIN VARIABEL KE CLASS

```python
# ========================================
# MAIN LOOP
# ========================================
if __name__ == "__main__":
    os.system('cls' if os.name == 'nt' else 'clear')
    print("=== INPUT SPESIFIKASI JEMBATAN ===")

    try:
        nama = input("Nama Jembatan [Jembatan Ampera]: ") or "Jembatan Ampera"
        kapasitas = float(input("Kapasitas Maksimal (Ton) [100]: ") or 100)
        panjang_m = float(input("Panjang Jembatan (Meter) [1200]: ") or 1200)

        # Input Kesehatan Struktur
        kesehatan_awal = float(input("Kesehatan Struktur Awal (%) [100]: ") or 100)

        b_waspada = float(input("Batas Waspada (%) [70]: ") or 70)
        b_kritis = float(input("Batas Kritis (%) [90]: ") or 90)
        f_momen = float(input("Faktor Momen Lentur (x) [1.5]: ") or 1.5)
        l_fatigue = float(input("Laju Fatigue (%/ton) [0.5]: ") or 0.5)
    except ValueError:
        print("Input tidak valid! Menggunakan nilai default.")
        nama, kapasitas, panjang_m = "Default Bridge", 100, 1200
        kesehatan_awal = 100 # TAMBAHAN BARU: Default jika error
        b_waspada, b_kritis, f_momen, l_fatigue = 70, 90, 1.5, 0.5

    # Menyesuaikan parameter instansiasi dengan variabel baru
    jembatan = Jembatan(nama, kapasitas, panjang_m, kesehatan_awal, b_waspada, b_kritis, f_momen, l_fatigue)
```

CLASS

```python
# ========================================
# CLASS JEMBATAN (LOGIKA INTI / BACKEND): Menerapkan OOP (Object-Oriented Programming)
# ========================================
class Jembatan:
    #Konstruktor adalah fungsi pertama yang dijalankan saat objek dibuat.
    #Fungsi __init__ menerima input manual dari pengguna dan menyimpannya sebagai state (keadaan) awal.
    def __init__(self, nama, kapasitas_maks, panjang_meter, kesehatan_awal, batas_waspada, batas_kritis, faktor_momen, laju_fatigue):
        # 1. Spesifikasi
        #Menerima 8 parameter input dari pengguna dan menyimpannya sebagai atribut self.
        self.nama = nama
        self.kapasitas_maks = kapasitas_maks
        self.panjang_meter = panjang_meter
        self.kesehatan_struktur = kesehatan_awal
        self.batas_waspada = batas_waspada
        self.batas_kritis = batas_kritis
        self.faktor_momen = faktor_momen
        self.laju_fatigue = laju_fatigue

        # 2. State
        self.beban_saat_ini = 0.0
        self.status = "NORMAL"
        self.warna_status = "green"
        self.palang_tertutup = False
        self.is_runtuh = False
        self.kendaraan_aktif = []
        self.kendaraan_ditolak = 0
        self.logs = ["[SISTEM] Simulasi dimulai."]
```

CLASS

```
#Algoritma Fixed-Size Bufer (Rolling Log) dengan menyimpan 3 log terakhir
# secara bergantian agar layar terminal tidak dipenuhi teks
def tambah_log(self, pesan):
    self.logs.insert(0, pesan)
    if len(self.logs) > 3:
        self.logs.pop()
```

```
def update_fsm_dan_palang(self):
    # Logika Tutup Permanen
    if self.kesehatan_struktur <= 0 and not self.is_runtuh:
        self.is_runtuh = True
        self.status = "RUNTUH"
        self.warna_status = "red"
        self.palang_tertutup = True
        self.tambah_log("💀 JEMBATAN RUNTUH! DITUTUP PERMANEN.")
        return # Langsung keluar, tidak perlu hitung FSM normal

    if self.is_runtuh:
        return # Jika sudah runtuh, hentikan evaluasi status

    #Menghitung kapasitas real-time seiring degradasi material.
    #Jika jembatan sudah tua/rusak (kesehatan_struktur rendah), kapasitas efektifnya menurun.
    #Lalu dihitung persentase beban saat ini. Palang akan turun ketika sudah tidak aman.
    kapasitas_efektif = self.kapasitas_maks * (self.kesehatan_struktur / 100)
    persentase = (self.beban_saat_ini / kapasitas_efektif) * 100 if kapasitas_efektif > 0 else 100
```

CLASS

```
# 1. FSM (Finite State Machine): Perubahan status dan warna UI dipicu oleh ambang batas (threshold) persentase.
if persentase >= self.batas_kritis:
    self.status = "KRITIS"
    self.warna_status = "red"
elif persentase >= self.batas_waspada:
    self.status = "WASPADA"
    self.warna_status = "yellow"
else:
    self.status = "NORMAL"
    self.warna_status = "green"

# 2. Logika Hysteresis Palang: Palang turun jika kritis. Palang hanya akan naik jika beban turun drastis di bawah 50%.
if self.status == "KRITIS" and not self.palang_tertutup:
    self.palang_tertutup = True
    self.tambah_log("🚨 KRITIS! PALANG DITUTUP.")

# Palang hanya buka jika < 50% (Aman)
elif self.palang_tertutup and persentase < 50:
    self.palang_tertutup = False
    self.tambah_log("🟢 AMAN. PALANG DIBUKA.")
```

CLASS

```
#Jika kendaraan berada di 20% area tengah jembatan, berat aslinya dikalikan dengan faktor_momen (misal 1.5x).
#Jika di area pilar (pinggir), beratnya normal.
#Momen Lentur (dan tegangan/stress) mencapai titik MAKSIMAL ketika beban berada tepat di tengah bentang.
# Jika beban berada di dekat pilar (pinggir), momen lenturnya mendekati nol (beban langsung disalurkan ke tanah).
def hitung_beban_fisika(self, berat_asli, posisi_meter):
    titik_tengah = self.panjang_meter / 2
    if abs(posisi_meter - titik_tengah) < (self.panjang_meter * 0.2):
        return berat_asli * self.faktor_momen
    return berat_asli

#Hitung ulang beban total secara berkala lewat looping
def hitung_ulang_total_beban(self):
    self.beban_saat_ini = sum(
        self.hitung_beban_fisika(k["berat"], k["posisi_meter"]) for k in self.kendaraan_aktif
    )
    self.update_fsm_dan_palang()
```

CLASS

```
#Jika kendaraan berada di 20% area tengah jembatan, berat aslinya dikalikan dengan faktor_momen (misal 1.5x).
#Jika di area pilar (pinggir), beratnya normal.
#Momen Lentur (dan tegangan/stress) mencapai titik MAKSIMAL ketika beban berada tepat di tengah bentang.
# Jika beban berada di dekat pilar (pinggir), momen lenturnya mendekati nol (beban langsung disalurkan ke tanah).
def hitung_beban_fisika(self, berat_asli, posisi_meter):
    titik_tengah = self.panjang_meter / 2
    if abs(posisi_meter - titik_tengah) < (self.panjang_meter * 0.2):
        return berat_asli * self.faktor_momen
    return berat_asli

#Hitung ulang beban total secara berkala lewat looping
def hitung_ulang_total_beban(self):
    self.beban_saat_ini = sum(
        self.hitung_beban_fisika(k["berat"], k["posisi_meter"]) for k in self.kendaraan_aktif
    )
    self.update_fsm_dan_palang()
```

CLASS

```python
def datang_kendaraan(self):
    #  Tolak semua kendaraan jika jembatan sudah runtuh
    if self.is_runtuh:
        self.kendaraan_ditolak += 1
        return

    #Jika palang sedang ditutup oleh sistem FSM, kendaraan yang datang otomatis ditolak dan dihitung.
    if self.palang_tertutup:
        self.kendaraan_ditolak += 1
        if random.random() < 0.2:  # Hanya log 20% dari total yang ditolak
            self.tambah_log("🚫 Kendaraan ditolak! Palang tertutup.")
        return

    # Probabilitas kedatangan 80%
    if random.random() < 0.8:
        jenis = random.choice(["Motor", "Mobil", "Truk"])
        #Data selang berat motor, mobil, dan truk didapat dari data berat tiap kendaraan di Indonesia
        if jenis == "Motor":
            berat = random.uniform(0.1, 0.3)
        elif jenis == "Mobil":
            berat = random.uniform(1.5, 3.0)
        else:
            berat = random.uniform(10.0, 25.0)

        # Kendaraan akan berada di jalur mana?
        # Jalur A (0->100) atau B (100->0)
        jalur = random.choice(["A", "B"])
        posisi = 0.0 if jalur == "A" else self.panjang_meter

        self.kendaraan_aktif.append({"jenis": jenis, "berat": berat, "posisi_meter": posisi, "jalur": jalur})
        self.tambah_log(f"➕ {jenis} ({berat:.1f}t) masuk Jalur {jalur}.")
```

CLASS

```
def update_pergerakan(self):
    kecepatan = self.panjang_meter / 15  # Kendaraan butuh ~15 tick untuk menyeberang
    for k in self.kendaraan_aktif[:]:
        if k["jalur"] == "A":
            k["posisi_meter"] += kecepatan
        else:
            k["posisi_meter"] -= kecepatan

        # Jika keluar jembatan, kendaraan tidak dihitung lagi
        if (k["jalur"] == "A" and k["posisi_meter"] > self.panjang_meter) or \
                (k["jalur"] == "B" and k["posisi_meter"] < 0):
            self.kendaraan_aktif.remove(k)
            # Kurangi kesehatan (Fatigue)
            self.kesehatan_struktur -= (k["berat"] / self.kapasitas_maks) * self.laju_fatigue
            if self.kesehatan_struktur < 0:
                self.kesehatan_struktur = 0

    self.hitung_ulang_total_beban()
```

KESIMPULAN

01

Bridge Capacity Management System (BCMS) membuktikan bahwa pemantauan jembatan dua arah dapat disimulasikan secara efisien dan otomatis tanpa campur tangan manual.

02

Logika Finite State Machine (FSM) mampu mengklasifikasikan status keamanan jembatan (Normal, Waspada, Kritis) dan mencegah kegagalan struktur akibat kelebihan beban (overload) melalui pembatasan akses dinamis.

03

Integrasi perhitungan beban statis dan akumulasi kelelahan material (fatigue) memperpanjang estimasi umur pakai jembatan serta memitigasi risiko keruntuhan dini.

BCMS MEMBUKTIKAN INTEGRASI LOGIKA KOMPUTASI MAMPU MENJAGA KETAHANAN INFRASTRUKTUR FISIK SECARA ADAPTIF.



